

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE SOFTWARE

Informe de Investigación y Práctica Experimental

Unidad II — Herramientas de Programación Web del Servidor

Programación del lado del servidor con PHP, PERL, Java/JSP y ASP.NET,
e implementación de una aplicación web segura (Clínica Veterinaria PFC)

Asignatura:

Aplicaciones Web — 5to Nivel «A»

Docente:

Ing. Gleiston Cicerón Guerrero Ulloa

Integrantes:

Johan Stalin Carvajal Loor
Michael Xavier Fajardo Montes
Jaime Josué Mariscal Cabrera

Repositorio GitHub:

<https://github.com/Grinjoww/GA-PractExperimentalUnidad-II>

Período Académico 2026–2027 PPA
Quevedo — Los Ríos — Ecuador

1 Introducción

La programación del lado del servidor es una parte esencial del desarrollo web, porque permite que una aplicación no se limite a mostrar páginas estáticas. En este tipo de programación, el servidor recibe las peticiones del navegador, procesa la información, valida los datos ingresados por el usuario, consulta o actualiza la base de datos y finalmente devuelve una respuesta. Por esa razón, el trabajo del backend se relaciona directamente con la lógica del negocio, la seguridad, el manejo de sesiones y la persistencia de información.

En la Unidad II se revisan varias herramientas utilizadas para construir aplicaciones web del lado del servidor. El informe se organiza en tres temas principales. En el primer tema se explican los conceptos generales de la programación del lado del servidor y se analizan tecnologías como PERL y PHP. En el segundo tema se aborda el desarrollo web con Java, considerando Servlets, JSP y Spring Boot. Finalmente, el tercer tema presenta aspectos importantes de ASP.NET Core, como su estructura, Razor y los mecanismos de seguridad que ofrece para aplicaciones modernas.

Como parte práctica, se desarrolló una aplicación web para una clínica veterinaria, relacionada con el Proyecto Fin de Curso. Esta aplicación fue implementada en dos tecnologías: PHP 8.2 con PDO y Java/Spring Boot 3 con JdbcTemplate. En ambas versiones se trabajó con autenticación de usuarios, protección de rutas y un módulo CRUD para la entidad Mascota. Además, se aplicaron controles de seguridad relacionados con OWASP, como el uso de tokens CSRF, almacenamiento seguro de contraseñas, saneamiento de salidas y cabeceras HTTP de seguridad. De esta manera, el informe no solo presenta teoría, sino también evidencia del sistema funcionando mediante capturas, fragmentos de código y una comparación técnica entre ambas tecnologías.

2 Tema 1: Introducción a la Programación del Lado del Servidor

2.1 Programación del lado del servidor: conceptos generales

La arquitectura cliente-servidor permite separar la parte visual de una aplicación de la parte encargada de procesar datos y reglas de negocio. El cliente, normalmente el navegador, se encarga de mostrar la interfaz y recibir las acciones del usuario. El servidor, en cambio, procesa la solicitud, ejecuta validaciones, accede a la base de datos y genera una respuesta. Esta separación facilita el mantenimiento, porque cada parte puede evolucionar sin afectar completamente a la otra. Además, permite que la lógica principal del sistema se mantenga centralizada en el backend, lo que resulta útil para controlar la seguridad y la consistencia de los datos [3].

En los últimos años, muchas aplicaciones web han adoptado arquitecturas basadas en API. Este enfoque permite que el frontend y el backend se comuniquen por medio de servicios, generalmente usando HTTP y formatos como JSON. Una API evita que el cliente tenga contacto directo con la base de datos y sirve como una capa intermedia para recuperar, registrar, actualizar o eliminar información. Las API RESTful son comunes porque trabajan con métodos HTTP conocidos, como GET, POST, PUT y DELETE, y ayudan a organizar las operaciones de manera clara [2].

Otro concepto importante en el desarrollo web del lado del servidor es el patrón MVC. Este patrón divide la aplicación en Modelo, Vista y Controlador. El Modelo representa los datos y las reglas del negocio; la Vista presenta la información al usuario; y el Controlador recibe las peticiones y decide qué operación debe ejecutarse. Esta forma de organizar el código ayuda a que el proyecto sea más ordenado, más fácil de probar y más sencillo de mantener. Lenguajes como PHP, Java y C# han adoptado este patrón mediante frameworks que permiten desarrollar aplicaciones de manera más estructurada [3].

2.2 PERL para aplicaciones web

PERL, cuyo nombre viene de *Practical Extraction and Report Language*, es un lenguaje de scripting creado por Larry Wall. Se caracteriza por ser flexible, portable y útil para procesar texto. A diferencia de lenguajes compilados, PERL se ejecuta mediante un intérprete, lo que permite escribir y probar código con rapidez. Aunque originalmente se usó mucho para tareas de administración y generación de reportes, también tuvo un papel importante en los primeros años de la web dinámica [1].

Una de las razones por las que PERL se volvió popular fue su disponibilidad en varios sistemas operativos, como Linux, UNIX, Windows y macOS. También cuenta con CPAN, un repositorio amplio de módulos y documentación que permitió ampliar sus capacidades. En su momento, esto ayudó a que muchos desarrolladores lo usaran para automatizar tareas, procesar archivos y construir scripts web [1].

En el desarrollo web, PERL fue muy usado junto con CGI. El modelo CGI permite que un servidor web ejecute un programa externo para generar contenido dinámico. El servidor entrega la información de la petición mediante variables de entorno y entrada estándar, mientras que el programa responde con cabeceras HTTP y contenido HTML. Sin embargo, este modelo tenía una limitación importante: normalmente se creaba un proceso nuevo por cada petición, lo que aumentaba el consumo de recursos cuando había muchos usuarios. Por esa razón, CGI fue perdiendo espacio frente a soluciones más eficientes.

Actualmente, PERL ya no se usa de forma tan frecuente para crear nuevas aplicaciones web grandes, pero sigue teniendo valor en áreas como administración de sistemas, bioinformática y procesamiento de texto. Desde el punto de vista académico, también es útil para entender cómo funcionaban las primeras aplicaciones dinámicas y cómo evolucionaron los modelos de integración entre servidores web y lenguajes de programación.

2.3 PHP: Hypertext Preprocessor

PHP es un lenguaje de scripting del lado del servidor pensado especialmente para el desarrollo web. Su facilidad para integrarse con HTML y bases de datos hizo que se convirtiera en una de las tecnologías más usadas para crear sitios dinámicos. A diferencia de otros lenguajes que luego fueron adaptados a la web, PHP nació con ese propósito, por lo que muchas de sus funciones están orientadas directamente al manejo de formularios, sesiones, archivos y consultas a bases de datos [3].

Con el tiempo, PHP ha evolucionado bastante. Actualmente cuenta con frameworks como Laravel, que permiten organizar el código siguiendo patrones como MVC, trabajar con rutas, usar middleware, validar datos y desarrollar APIs de manera más limpia. Laravel también ofrece herramientas como Eloquent ORM, que facilita el trabajo con bases de datos mediante clases y modelos. Además, proporciona soluciones para autenticación, autorización y protección de rutas [2].

En proyectos pequeños y medianos, PHP suele ser una opción práctica porque permite comenzar rápido y no exige una configuración demasiado compleja. También tiene una comunidad grande, abundante documentación y una presencia fuerte en servicios de hosting compartido. Esto es importante en contextos donde se busca un despliegue económico y accesible. Sin embargo, en proyectos grandes, si no se aplican buenas prácticas, el código puede volverse difícil de mantener. Por eso, aunque PHP sea sencillo de iniciar, se recomienda trabajar con una estructura clara, validaciones del lado del servidor y acceso a datos seguro.

En esta práctica se utilizó PHP 8.2 con PDO. PDO permite conectarse a la base de datos y ejecutar consultas preparadas, lo que evita construir SQL mediante concatenación directa de datos ingresados por el usuario. Esta decisión fue importante para cumplir con las medidas de seguridad solicitadas y para reducir el riesgo de inyección SQL.

2.4 Comparativa tecnológica y criterios de selección

La elección de una tecnología de servidor no debería basarse solo en gustos personales. En un proyecto real conviene analizar aspectos como facilidad de aprendizaje, rendimiento, seguridad, disponibilidad de hosting, comunidad, documentación, escalabilidad y facilidad para realizar pruebas. Un estudio comparativo sobre proyectos con PHP y Java muestra que PHP suele requerir menos código para operaciones CRUD básicas, mientras que Java ofrece ventajas cuando el sistema crece y necesita mayor organización, pruebas e integración empresarial [3].

PHP destaca por su rapidez para iniciar un proyecto. Su sintaxis es más directa y permite construir formularios, conexiones a base de datos y páginas dinámicas con menor configuración. Esto lo hace atractivo para prototipos, proyectos académicos y soluciones que deben publicarse en hosting económico. En cambio, Java con Spring Boot requiere más configuración inicial y mayor conocimiento de conceptos como inyección de dependencias, anotaciones y contenedores de inversión de control. Aun así, una vez configurado, Spring Boot ofrece una estructura sólida para sistemas más grandes.

En cuanto al rendimiento, PHP puede responder muy bien en operaciones simples, especialmente cuando se usa PHP-FPM y OPcache. Java, por su parte, aprovecha la JVM y suele manejar mejor escenarios de concurrencia sostenida o lógica más compleja. En seguridad, Spring Security incluye varias protecciones por defecto, como CSRF, control de sesiones y cabeceras de seguridad. En PHP, muchas de estas medidas deben implementarse manualmente, lo cual da más control, pero también aumenta la responsabilidad del desarrollador.

Criterio	PHP + Laravel	Java + Spring/JSP
Tiempo de desarrollo inicial	Más rápido para formularios, CRUD y prototipos.	Más lento al inicio por su configuración y estructura.

Criterio	PHP + Laravel	Java + Spring/JSP
Curva de aprendizaje	Baja; resulta accesible para estudiantes y proyectos pequeños.	Media-alta; exige más dominio de POO y arquitectura.
Rendimiento	Bueno en operaciones web comunes y consultas simples.	Muy sólido en procesos grandes y concurrencia sostenida.
Reutilización de código	Buena si se trabaja con patrones y frameworks.	Alta por el uso de capas, servicios e inyección de dependencias.
Testabilidad	Correcta con PHPUnit y separación de responsabilidades.	Más fuerte por sus herramientas de pruebas e integración.
Escalabilidad	Adecuada, aunque requiere disciplina en la estructura.	Muy buena para sistemas empresariales y de largo plazo.
Comunidad y ecosistema	Muy amplia, con bastante documentación y hosting accesible.	Madura, fuerte en empresas y aplicaciones robustas.

Cuadro 1: Criterios de selección entre PHP+Laravel y Java+Spring/JSP, según [3].

En resumen, PHP resulta conveniente cuando se busca rapidez, bajo costo y facilidad de despliegue. Java/Spring Boot, en cambio, es una alternativa más fuerte cuando se requiere una arquitectura más robusta, mayor control de seguridad por defecto y mejor soporte para proyectos que pueden crecer con el tiempo.

3 Tema 2: Desarrollo Web con Java y JSP

3.1 Java para el desarrollo web: plataforma y herramientas

Java no se limita al desarrollo de aplicaciones de escritorio. También puede utilizarse en el lado del servidor para recibir peticiones HTTP, procesarlas y devolver respuestas al navegador. En una aplicación web Java, el cliente realiza una solicitud y el servidor ejecuta componentes encargados de responder con HTML, datos o redirecciones. Este modelo mantiene la lógica principal en el backend y facilita el control de acceso, la conexión a bases de datos y la organización del código [4].

La plataforma Java EE, actualmente Jakarta EE, fue diseñada para aplicaciones empresariales y web. Dentro de este ecosistema se encuentran los Servlets, que son clases Java que atienden peticiones HTTP; las JSP, que permiten generar páginas dinámicas combinando HTML con elementos Java; y JPA, que se usa para trabajar con datos de una forma más orientada a objetos. Aunque muchas aplicaciones modernas usan frameworks como Spring Boot, estos conceptos siguen siendo importantes porque ayudan a comprender la base del desarrollo web en Java.

Para trabajar con Java en la web se necesitan varias herramientas. El JDK permite compilar y ejecutar código Java. Un contenedor como Tomcat se encarga de ejecutar Servlets y JSP. También se utiliza un IDE como IntelliJ IDEA, Eclipse o VS Code con extensiones, que facilitan la escritura, organización y ejecución del proyecto. En la práctica actual se utilizó Spring Boot porque simplifica la configuración y permite ejecutar la aplicación con un servidor embebido.

3.2 Java Servlets

Un Servlet es una clase Java que se ejecuta en el servidor y responde a peticiones web. Puede recibir parámetros, consultar una base de datos, manejar sesiones y construir una respuesta. En sus primeras etapas, los Servlets fueron una solución importante porque permitían crear contenido dinámico sin depender de scripts externos como CGI. Basham, Sierra y Bates explican que el Servlet es uno de los componentes centrales de la web en Java, ya que actúa como intermediario entre el navegador y la lógica del servidor [5].

El ciclo de vida de un Servlet tiene tres momentos principales. Primero se ejecuta `init()`, usado para preparar recursos. Luego se ejecuta `service()`, que atiende cada solicitud y decide si debe usar `doGet()` o `doPost()`. Finalmente, `destroy()` se ejecuta cuando el servidor libera el Servlet. Esta estructura permite que el servidor gestione los objetos de manera más eficiente.

```
1 @WebServlet("/saludo")
2 public class SaludoServlet extends HttpServlet {
3     protected void doGet(HttpServletRequest req,
4                           HttpServletResponse res)
5         throws ServletException, IOException {
6         String nombre = req.getParameter("nombre");
7         res.setContentType("text/html");
8         PrintWriter out = res.getWriter();
9         out.println("<h1>Hola, " + nombre + "!</h1>");
10    }
11 }
```

Listing 1: Servlet que responde un saludo

El manejo de sesiones también es importante, porque HTTP no conserva memoria entre peticiones. Por eso, Java permite guardar información temporal del usuario mediante `HttpSession`. Esto sirve, por ejemplo, para mantener activo un inicio de sesión o recordar datos mientras el usuario navega por el sistema.

```
1 HttpSession session = request.getSession();
2 session.setAttribute("usuario", "Carlos");
3 String u = (String) session.getAttribute("usuario");
```

Listing 2: Manejo de sesión en un Servlet

3.3 JavaServer Pages (JSP)

JSP nació para facilitar la creación de páginas dinámicas en Java. Cuando se genera HTML directamente desde un Servlet, el código puede volverse extenso y difícil de leer. JSP permite escribir una página principalmente en HTML e incluir elementos Java cuando se necesita mostrar datos dinámicos. Esto ayuda a separar mejor la presentación de la lógica del sistema [5].

Cuando el servidor recibe una petición hacia un archivo JSP, Tomcat lo transforma internamente en un Servlet, lo compila y lo ejecuta. La primera carga puede tardar un poco más, pero luego el Servlet generado queda disponible para nuevas peticiones. Entre sus elementos más conocidos están las expresiones, los scriptlets y las directivas. Aunque actualmente se recomienda evitar demasiada lógica dentro de las vistas, JSP sigue siendo útil para comprender cómo se construyen páginas dinámicas en Java.

```
1 <%@ page import="java.util.List, java.util.ArrayList" %>
2 <!DOCTYPE html><html><body>
3 <h1>Productos</h1>
4 <%
5     List<String> lista = new ArrayList<>();
6     lista.add("Laptop"); lista.add("Mouse");
7 %>
8 <ul>
```

```
9 <% for (String item : lista) { %>
10     <li><%= item %></li>
11 <% } %>
12 </ul>
13 </body></html>
```

Listing 3: Página JSP con lista dinámica

En la práctica, JSP y Servlets se suelen relacionar con el patrón MVC. El Servlet funciona como controlador, las clases Java representan el modelo y la JSP se encarga de la vista. Esta separación permite que el código sea más entendible y mantenible.

3.4 Introducción a Spring Boot para desarrollo web

Spring Boot es una extensión del ecosistema Spring que reduce gran parte de la configuración necesaria para crear aplicaciones web Java. Antes, crear un proyecto con Spring podía requerir varios archivos XML y configuraciones manuales. Spring Boot simplifica este proceso mediante autoconfiguración, dependencias listas para usar y un servidor embebido, normalmente Tomcat [6].

Entre sus ventajas se encuentra el uso de *starters*, que agrupan dependencias según el tipo de proyecto. Por ejemplo, `spring-boot-starter-web` permite crear controladores web, mientras que `spring-boot-starter-security` incorpora herramientas de autenticación y autorización. También se puede usar Spring Initializr para generar rápidamente la estructura base del proyecto.

```
1 mi-proyecto/
2   src/main/java/com/uteq/
3     MiApp.java
4   src/main/resources/
5     application.properties
6   pom.xml
```

Listing 4: Estructura de un proyecto Spring Boot

Crear una ruta en Spring Boot es bastante directo. Un controlador puede recibir una petición y devolver una respuesta sin necesidad de configurar un servidor externo.

```
1 @RestController
2 public class HolaController {
3     @GetMapping("/hola")
4     public String hola() {
5         return "Hola desde Spring Boot!";
6     }
7     @GetMapping("/saludo/{nombre}")
8     public String saludo(@PathVariable String nombre) {
9         return "Bienvenido, " + nombre;
10    }
11 }
```

Listing 5: Controlador REST en Spring Boot

Spring Boot fue elegido como segunda tecnología porque permite comparar una solución más manual en PHP con una plataforma que integra muchas funciones listas para producción, especialmente en seguridad y estructura del proyecto.

4 Tema 3: Desarrollo Web con ASP.NET

La plataforma .NET de Microsoft es una alternativa importante para construir aplicaciones web modernas. ASP.NET Core permite crear páginas web, APIs y servicios que pueden ejecutarse en Windows, Linux o macOS. En esta unidad se revisa como parte de las herramientas del

lado del servidor, aunque en la práctica se eligió Java/Spring Boot como segunda tecnología de implementación. Aun así, conocer ASP.NET Core permite comparar distintos ecosistemas usados en el desarrollo profesional [7], [8].

4.1 La plataforma .NET y ASP.NET

.NET ha cambiado bastante desde sus primeras versiones. Durante muchos años se trabajó con .NET Framework, orientado principalmente a Windows. Luego apareció .NET Core, con un enfoque multiplataforma y más ligero. Con .NET 5 se unificó el ecosistema y las versiones posteriores, como .NET 6 y .NET 8, reforzaron el soporte para aplicaciones web, nube, contenedores y servicios modernos [7].

ASP.NET Core es el framework web de esta plataforma. Permite trabajar con MVC, Razor Pages, Web APIs y Blazor. Su diseño modular facilita agregar solo los componentes necesarios para cada proyecto. Además, puede ejecutarse sobre Kestrel, un servidor web ligero que suele integrarse con otros servidores como IIS o Nginx en ambientes de producción. Esta flexibilidad lo vuelve útil tanto para aplicaciones pequeñas como para sistemas empresariales.

En seguridad, ASP.NET Core ofrece varias protecciones integradas. Por ejemplo, cuenta con mecanismos para autenticación, autorización, validación de formularios y prevención de ataques comunes como XSS y CSRF. También se integra con soluciones basadas en OAuth, JWT e identidad de usuarios. Estas características hacen que sea una tecnología fuerte cuando se busca construir aplicaciones con una base de seguridad desde el inicio [12], [13].

4.2 ASP.NET Core: estructura de un proyecto web

Un proyecto ASP.NET Core se organiza normalmente en carpetas que separan responsabilidades. En una aplicación MVC se encuentran carpetas como **Models**, **Views** y **Controllers**. Esta separación ayuda a mantener el orden del sistema, ya que cada parte cumple una función específica. Los modelos representan datos, las vistas muestran información y los controladores procesan las solicitudes.

Uno de los elementos más importantes en ASP.NET Core es el pipeline de middleware. El middleware funciona como una cadena de componentes por la que pasa cada solicitud. Por ejemplo, una petición puede pasar primero por redirección HTTPS, luego por archivos estáticos, después por enrutamiento, autenticación y autorización. El orden importa porque no tendría sentido autorizar a un usuario antes de autenticarlo [11].

La inyección de dependencias también es una característica central. ASP.NET Core permite registrar servicios con distintos tiempos de vida: *transient*, *scoped* y *singleton*. Esto ayuda a desacoplar clases y facilita las pruebas. En un proyecto con repositorios, por ejemplo, se puede registrar una interfaz y su implementación concreta para que el controlador no dependa directamente de una clase específica.

4.3 Razor Pages y Razor Views

Razor es el motor de vistas usado por ASP.NET Core para combinar HTML con código C#. Su sintaxis permite insertar variables, condiciones y ciclos dentro de una página sin perder completamente la legibilidad del HTML. Esto facilita la creación de páginas dinámicas del lado del servidor [11], [14].

En ASP.NET Core existen dos formas comunes de trabajar con Razor: Razor Views dentro de MVC y Razor Pages. MVC separa controladores y vistas, lo que resulta útil en proyectos con mayor complejidad. Razor Pages, en cambio, agrupa la vista y su lógica relacionada en una estructura más directa, por lo que puede ser más sencilla para módulos pequeños o formularios administrativos.

Ambos enfoques se usan dependiendo del contexto. MVC ofrece una separación más marcada y suele ser adecuado para sistemas grandes. Razor Pages puede ser conveniente cuando se necesita

construir pantallas de forma rápida y ordenada, sin crear demasiados archivos separados. En cualquier caso, Razor permite mantener una relación clara entre la interfaz y los datos que se muestran al usuario.

4.4 Formularios, validación y seguridad en ASP.NET

La seguridad en ASP.NET Core se trabaja desde varias capas. Una aplicación web debe validar los datos ingresados por el usuario, controlar quién puede acceder a cada recurso, proteger formularios y evitar que se ejecuten scripts no autorizados. Aunque la validación en el navegador ayuda a mejorar la experiencia, la validación realmente importante debe realizarse siempre en el servidor, porque el cliente puede ser manipulado [12].

Para prevenir CSRF, ASP.NET Core utiliza tokens antifalsificación. Estos tokens permiten verificar que un formulario fue enviado desde la propia aplicación y no desde un sitio externo. En MVC puede usarse el atributo `[ValidateAntiForgeryToken]`, mientras que en otros escenarios se configura el servicio correspondiente. Esta protección es importante en formularios de registro, edición o eliminación.

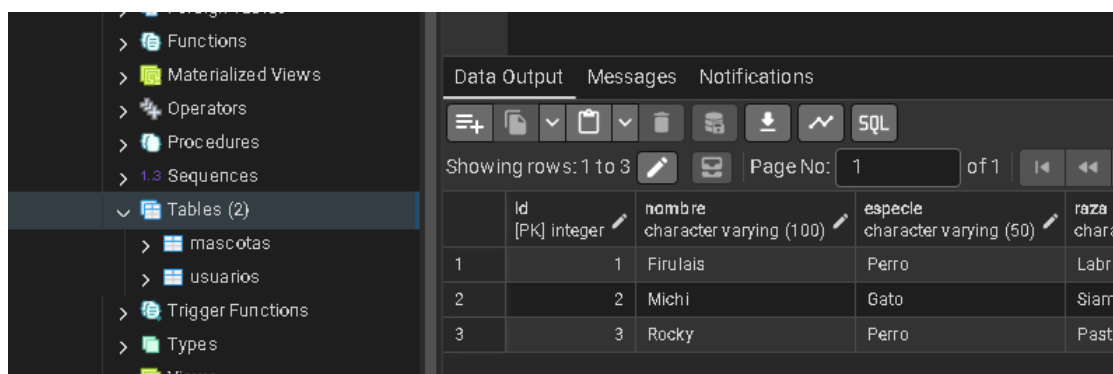
Frente a XSS, ASP.NET Core codifica por defecto muchos valores que se imprimen en las vistas. Aun así, el desarrollador debe tener cuidado con datos ingresados por los usuarios, especialmente si se van a mostrar nuevamente en la página. También se recomienda usar cabeceras de seguridad, HTTPS y políticas de contenido. En conjunto, estas medidas permiten construir una defensa en profundidad, donde varias capas ayudan a reducir el riesgo de ataques comunes.

5 Resultados de la Implementación (Proyecto)

Como aplicación práctica de la Unidad II, se desarrolló el backend de una clínica veterinaria en dos tecnologías: **PHP 8.2 + PDO** y **Java/Spring Boot 3 + JdbcTemplate**. En ambas versiones se implementó autenticación de usuarios, cierre de sesión, protección de rutas y un módulo CRUD para registrar, listar, editar y eliminar mascotas. Además, se aplicaron medidas de seguridad como consultas preparadas, tokens CSRF, almacenamiento seguro de contraseñas y cabeceras HTTP.

5.1 Configuración del entorno y base de datos

La base de datos `veterinaria_db` fue creada en PostgreSQL usando el archivo `schema_postgres.sql`. En ella se generaron las tablas `usuarios` y `mascotas`, necesarias para la autenticación y el módulo CRUD.



	Id [PK] integer	nombre character varying (100)	especie character varying (50)	raza character varying (50)
1	1	Firulais	Perro	Labra
2	2	Michi	Gato	Siam
3	3	Rocky	Perro	Pasto

Figura 1: Base de datos `veterinaria_db` con las tablas `usuarios` y `mascotas`.

Antes de ejecutar las pruebas visuales del sistema, se realizó el análisis estático del código PHP con PHPStan en nivel 5. El resultado obtenido fue sin errores, lo cual ayuda a comprobar

que la lógica principal del proyecto no presenta fallos detectados por la herramienta.

```
PS C:\Users\jaime\Desktop\practica-veterinaria\proyecto-veterinaria\php-veterinaria> composer install
No composer.lock file present. Updating dependencies to latest instead of installing from lock file. See https://getcomposer.org/install for more information.
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
- Locking phpstan/phpstan (1.12.33)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
- Downloading phpstan/phpstan (1.12.33)
- Installing phpstan/phpstan (1.12.33): Extracting archive
Generating autoload files
1 package you are using is looking for funding.
Use the `composer fund` command to find out more!
PS C:\Users\jaime\Desktop\practica-veterinaria\proyecto-veterinaria\php-veterinaria> composer phpstan
Note: Using configuration file C:\Users\jaime\Desktop\practica-veterinaria\proyecto-veterinaria\php-veterinaria\phpstan.neon.
11/11 [=====] 100%
[OK] No errors
```

Figura 2: Resultado del análisis estático con `composer phpstan`, mostrando la ejecución sin errores.

5.2 Autenticación segura en PHP

En la versión PHP se implementó un registro de usuarios donde las contraseñas se almacenan con `password_hash()` usando Argon2id. Para iniciar sesión se utiliza `password_verify()`, evitando comparar contraseñas en texto plano. Además, al autenticar al usuario se regenera el identificador de sesión para reducir el riesgo de fijación de sesión.

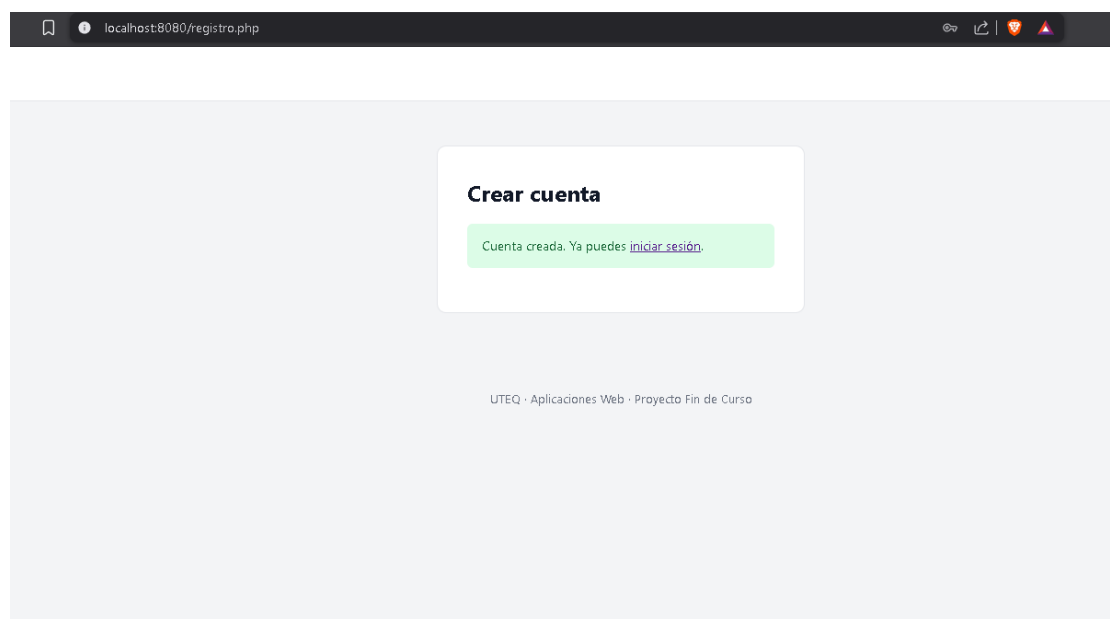


Figura 3: Registro exitoso de usuario en el sistema PHP.

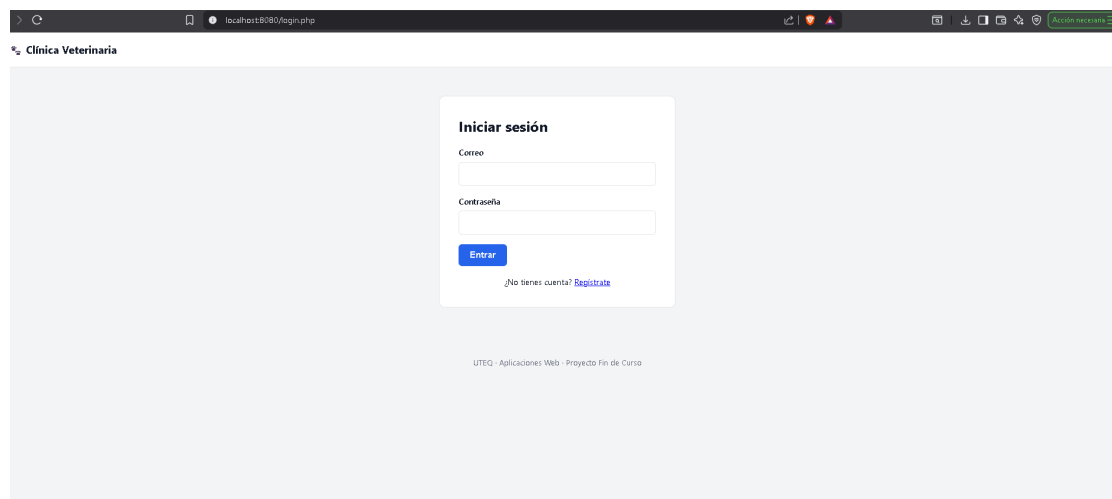


Figura 4: Pantalla de inicio de sesión del sistema PHP.

Cada formulario incluye un token CSRF validado en el servidor antes de procesar los datos. Esta medida evita que una petición enviada desde otro sitio pueda ejecutar acciones sin autorización del usuario.

```
1 public static function verificarCsrf(?string $token): bool
2 {
3     return !empty($_SESSION['csrf_token'])
4         && is_string($token)
5         && hash_equals($_SESSION['csrf_token'], $token);
6 }
```

Listing 6: Verificación del token CSRF en PHP

5.3 Módulo CRUD con patrón Repository y PDO

El módulo de mascotas fue construido usando el patrón Repository y PDO. Esta estructura permite separar el acceso a datos de la lógica de la aplicación. Las consultas se realizan con *prepared statements*, por lo que los datos del usuario no se concatenan directamente dentro del SQL.

```
1 $sql = 'INSERT INTO mascotas (nombre, especie, raza, edad, nombre_dueno,
2     telefono)
3     VALUES (:nombre, :especie, :raza, :edad, :dueno, :telefono)';
4 $stmt = $this->pdo->prepare($sql);
5 $stmt->bindValue(':nombre', $m->nombre, PDO::PARAM_STR);
6 $stmt->bindValue(':especie', $m->especie, PDO::PARAM_STR);
7 $stmt->bindValue(':edad', $m->edad, PDO::PARAM_INT);
8 $stmt->execute();
```

Listing 7: Inserción con prepared statements en PDO

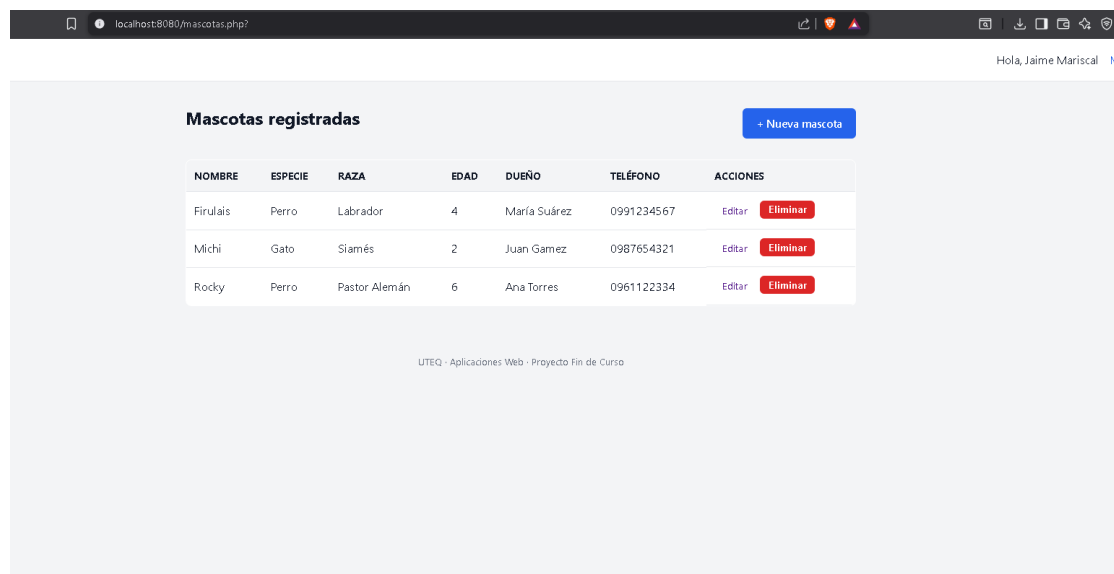


Figura 5: Listado de mascotas registradas en la versión PHP.

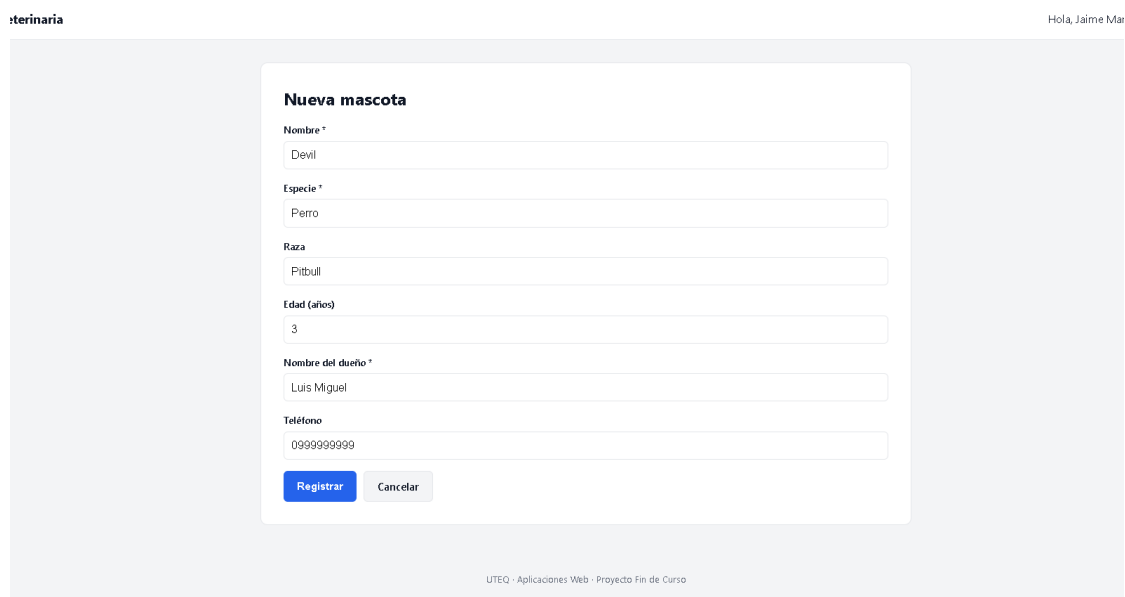


Figura 6: Formulario de creación de una nueva mascota en PHP.

El formulario 'Editar mascota' contiene los siguientes campos:

- Nombre ***: Input con el valor 'Firulais'.
- Especie ***: Input con el valor 'Perro'.
- Raza**: Input con el valor 'Labrador'.
- Edad (años)**: Input con el valor '4'.
- Nombre del dueño ***: Input con el valor 'María Suárez'.
- Teléfono**: Input con el valor '0991234567'.

En la parte inferior del formulario hay dos botones: 'Guardar cambios' (azul) y 'Cancelar' (gris). En la parte inferior de la pantalla se muestra el pie de página: 'UTEQ · Aplicaciones Web · Proyecto Fin de Curso'.

Figura 7: Formulario de edición de una mascota existente en PHP.

La interfaz muestra el título 'Mascotas registradas' y un botón '+ Nueva mascota'. Una barra de mensaje verde indica: 'Mascota eliminada correctamente.'.

NOMBRE	ESPECIE	RAZA	EDAD	DUEÑO	TELÉFONO	ACCIONES
Michi	Gato	Siamés	2	Juan Gamez	0987654321	Editar Eliminar

Figura 8: Confirmación de eliminación de una mascota en PHP.

5.4 Verificación de seguridad en PHP

La aplicación PHP incluye cabeceras HTTP de seguridad para reducir riesgos comunes en aplicaciones web. Entre ellas se encuentran `X-Frame-Options`, `X-Content-Type-Options`, `Content-Security-Policy` y `Referrer-Policy`. Estas cabeceras se verificaron desde las herramientas de desarrollo del navegador.

Name	×	Headers	Preview	Response	Initiator	Timing	Adblock	Cookies
login.php		Content-Type		text/html; charset=UTF-8				
estilos.css		Date		Sat, 20 Jun 2026 00:56:15 GMT				
		Expires		Thu, 19 Nov 1981 08:52:00 GMT				
		Host		localhost:8080				
		Pragma		no-cache				
		Referrer-Policy		no-referrer				
		X-Content-Type-Options		nosniff				
		X-Frame-Options		DENY				
		X-Powered-By		PHP/8.2.12				
		X-Xss-Protection		1; mode=block				
		▼ Request Headers		☐ Raw				
		Accept		text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8				
		Accept-Encoding		gzip, deflate, br, zstd				
		Accept-Language		es-ES,es;q=0.6				
		Cache-Control		max-age=0				
		Connection		keep-alive				
		Cookie		PHPSESSID=4vqc1rqdmave59e3813um5125j				
		Host		localhost:8080				
		Sec-Ch-Ua		"Brave";v="149", "Chromium";v="149", "Not)A;Brand";v="24"				
		Sec-Ch-Ua-Mobile		?0				
		Sec-Ch-Ua-Platform		"Windows"				
		Sec-Fetch-Dest		document				
		Sec-Fetch-Mode		navigate				
		Sec-Fetch-Site		same-origin				
		Sec-Fetch-User		?1				
		Sec-Gpc		1				
		Upgrade-Insecure-Requests		1				
		User-Agent		Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36				
2 requests		4.8 kB transferre						

Figura 9: Cabeceras de seguridad HTTP verificadas en la versión PHP desde la pestaña Red del navegador.

5.5 Segunda tecnología: Java / Spring Boot

La segunda implementación se desarrolló con Spring Boot 3 y JdbcTemplate. Esta versión mantiene la misma idea funcional de la aplicación PHP: autenticación de usuarios y módulo CRUD de mascotas. Spring Security aporta varias medidas de seguridad de forma integrada, como protección CSRF, control de acceso a rutas y manejo de sesión.

```

1 @Bean
2 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
3     http
4         .authorizeHttpRequests(auth -> auth
5             .requestMatchers("/", "/login", "/registro", "/css/**").permitAll()
6             .anyRequest().authenticated())
7         .formLogin(form -> form.loginPage("/login").defaultSuccessUrl("/mascotas", true))
8         .headers(headers -> headers
9             .frameOptions(HeadersConfigurer.FrameOptionsConfig::deny));
10     return http.build();
11 }

```

Listing 8: Configuración de seguridad en Spring Security

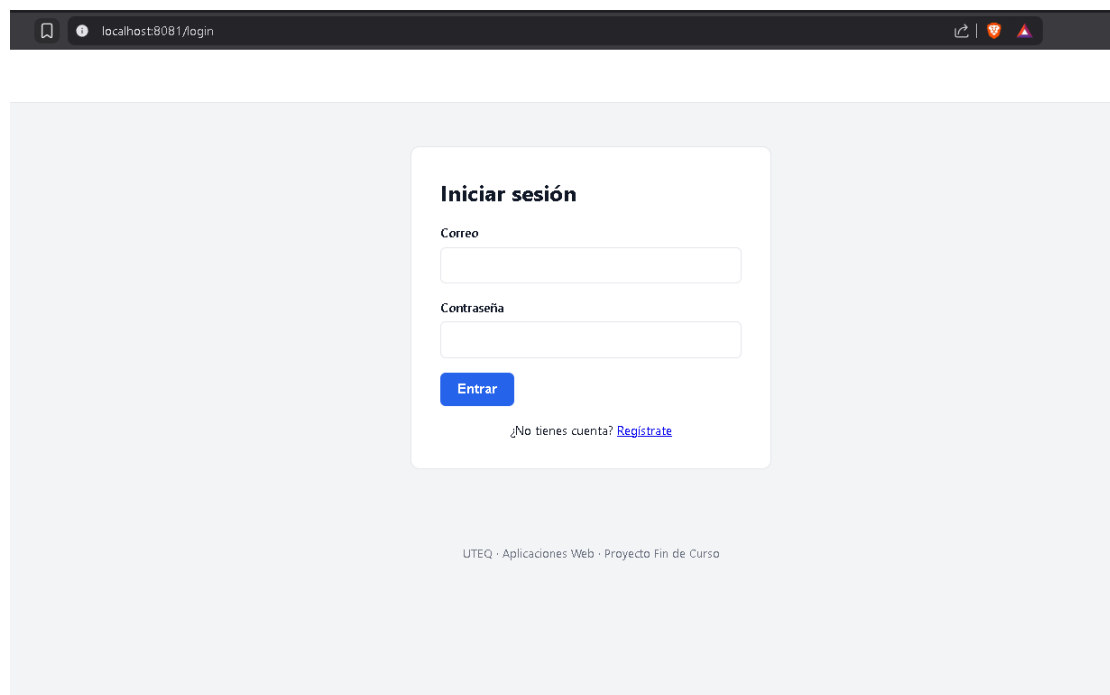


Figura 10: Pantalla de inicio de sesión del sistema Spring Boot.

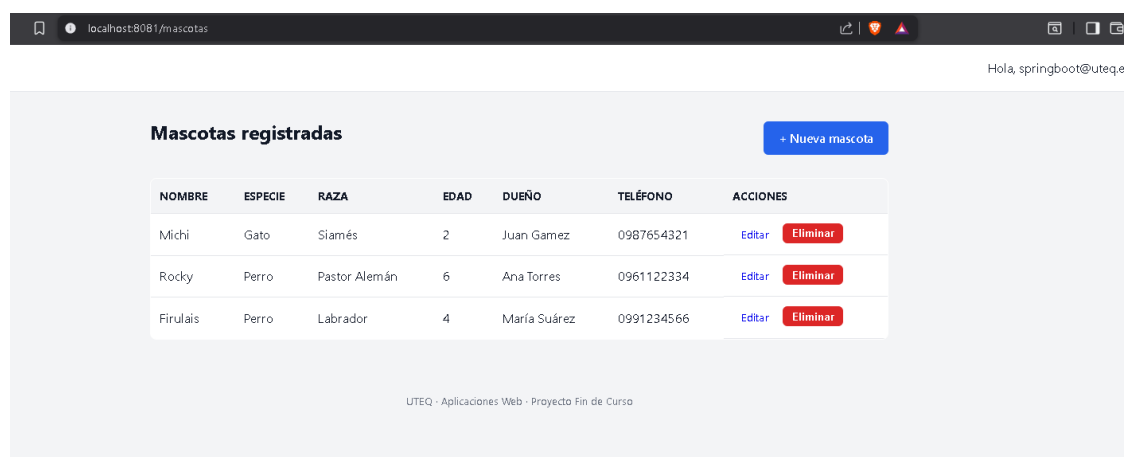


Figura 11: Listado de mascotas en la versión Spring Boot.

The screenshot shows a web browser at the URL `localhost:8081/mascotas/nueva`. The page title is "Nueva mascota". The form contains the following fields and values:

- Nombre ***:
- Especie ***:
- Raza**:
- Edad (años)**:
- Nombre del dueño ***:
- Teléfono**:

At the bottom of the form are two buttons: "Registrar" (blue) and "Cancelar" (gray). The footer of the page reads "UTEQ - Aplicaciones Web - Proyecto Fin de Curso".

Figura 12: Formulario de creación de mascota en Spring Boot.

The screenshot shows a web browser at the URL `localhost:8081/mascotas/editar/6`. The page title is "Editar mascota". The form contains the following fields and values:

- Nombre ***:
- Especie ***:
- Raza**:
- Edad (años)**:
- Nombre del dueño ***:
- Teléfono**:

At the bottom of the form are two buttons: "Guardar cambios" (blue) and "Cancelar" (gray). The footer of the page reads "UTEQ - Aplicaciones Web - Proyecto Fin de Curso".

Figura 13: Formulario de edición de mascota en Spring Boot.

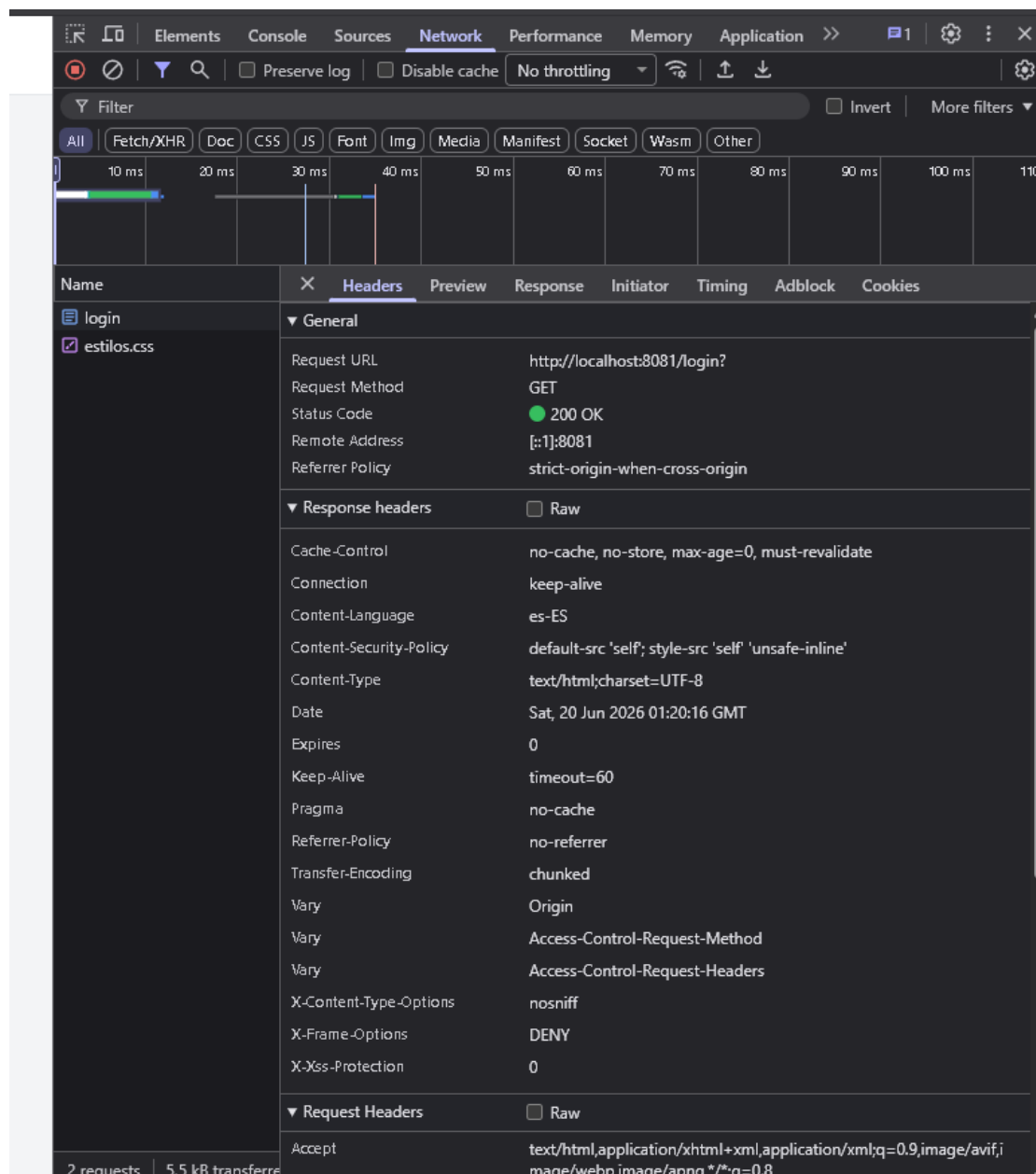


Figura 14: Cabeceras de seguridad HTTP verificadas en la versión Spring Boot.

5.6 Tabla comparativa de las dos tecnologías

La siguiente tabla compara las dos implementaciones del proyecto con criterios técnicos objetivos:

Criterio	PHP 8.2 + PDO	Spring Boot 3
Líneas de código (auth)	Menor; varias medidas se implementan manualmente.	Mayor configuración inicial, pero con seguridad integrada.
Curva de aprendizaje	Baja; la sintaxis es directa y fácil de probar.	Media-alta; requiere conocer anotaciones, DI y estructura Spring.

Criterio	PHP 8.2 + PDO	Spring Boot 3
Seguridad por defecto	Depende más del programador y de la configuración aplicada.	Más completa por defecto gracias a Spring Security.
Rendimiento	Bueno para operaciones CRUD y despliegues sencillos.	Muy bueno para concurrencia y proyectos de mayor tamaño.
Acceso a datos	PDO con consultas preparadas.	JdbcTemplate con PreparedStatement.
Hosting en Ecuador	Más accesible en hosting compartido.	Requiere normalmente VPS o servicios en la nube.
Facilidad de pruebas	Buena si se separa el código por capas.	Alta por las herramientas integradas del ecosistema Spring.
Cifrado de contraseñas	Argon2id mediante <code>password_hash()</code> .	BCrypt con factor de costo 12.

Cuadro 2: Comparativa técnica de las dos implementaciones del proyecto.

5.7 Repositorio y control de versiones

El código fuente fue gestionado con Git. Como parte de la evidencia, se presenta el historial de commits del repositorio, donde se refleja la participación de los integrantes del equipo.

Además del código, en el repositorio se incluyó una carpeta `docs/adr` con dos registros de decisiones de arquitectura (ADR). El primero explica por qué se eligió PHP como tecnología principal y Java/Spring Boot como segunda opción, considerando sobre todo el costo del *hosting* y la experiencia del equipo. El segundo justifica el uso de PostgreSQL como base de datos y el acceso a los datos mediante consultas preparadas. La idea de documentar estas decisiones fue dejar constancia del porqué de cada elección, y no solo del resultado final, que es algo que se suele perder cuando un proyecto avanza.



Figura 15: Historial de commits del repositorio Git con aportes de los integrantes del equipo.

6 Conclusiones

- La programación del lado del servidor permite que una aplicación web procese datos, maneje sesiones, valide entradas y conecte con una base de datos. Esto demuestra que el backend es una parte clave para construir sistemas dinámicos y seguros.
- PHP resultó una tecnología práctica para implementar rápidamente la autenticación y el CRUD del proyecto. Su integración con PDO permitió trabajar con consultas preparadas y reducir riesgos de inyección SQL.
- Java/Spring Boot presentó una estructura más formal y con mayor configuración inicial, pero también ofreció ventajas importantes en seguridad por defecto, organización del código y soporte para proyectos de mayor escala.
- La comparación entre PHP y Spring Boot permitió observar que no existe una única tecnología ideal para todos los casos. PHP puede ser más conveniente por facilidad y costo de despliegue, mientras que Spring Boot es más fuerte cuando se busca una arquitectura robusta y mantenible.
- La práctica permitió aplicar controles de seguridad relacionados con OWASP, como protección CSRF, saneamiento de salidas, almacenamiento seguro de contraseñas y cabeceras HTTP. Esto refuerza la importancia de considerar la seguridad desde el desarrollo y no solo al final del proyecto.

7 Anexo C — Preguntas de Análisis

Análisis: ¿cuál implementación resultó más segura por defecto?

De las dos implementaciones, **Java/Spring Boot resultó más segura «de fábrica»**. Sin escribir código adicional, Spring Security activa la protección CSRF, regenera el identificador de sesión tras la autenticación y permite configurar las cabeceras de seguridad con muy pocas líneas. En PHP, en cambio, cada uno de esos controles debió implementarse manualmente: el *token* CSRF con su verificación mediante `hash_equals()`, la llamada explícita a

`session_regenerate_id(true)` y el envío manual de las cabeceras de seguridad. Como contrapartida, PHP ofrece mayor transparencia —se observa exactamente qué hace cada control— pero traslada al desarrollador la responsabilidad de no omitir ninguna medida. En síntesis, funcionalidades que en PHP fueron manuales (CSRF, cabeceras, regeneración de sesión) venían incluidas en Spring; a la inversa, PHP no requirió configuración inicial para arrancar, mientras que Spring exigió definir el `SecurityFilterChain` y el `UserDetailsService`.

Evaluación: ¿qué tecnología recomendarían para Los Ríos y Ecuador?

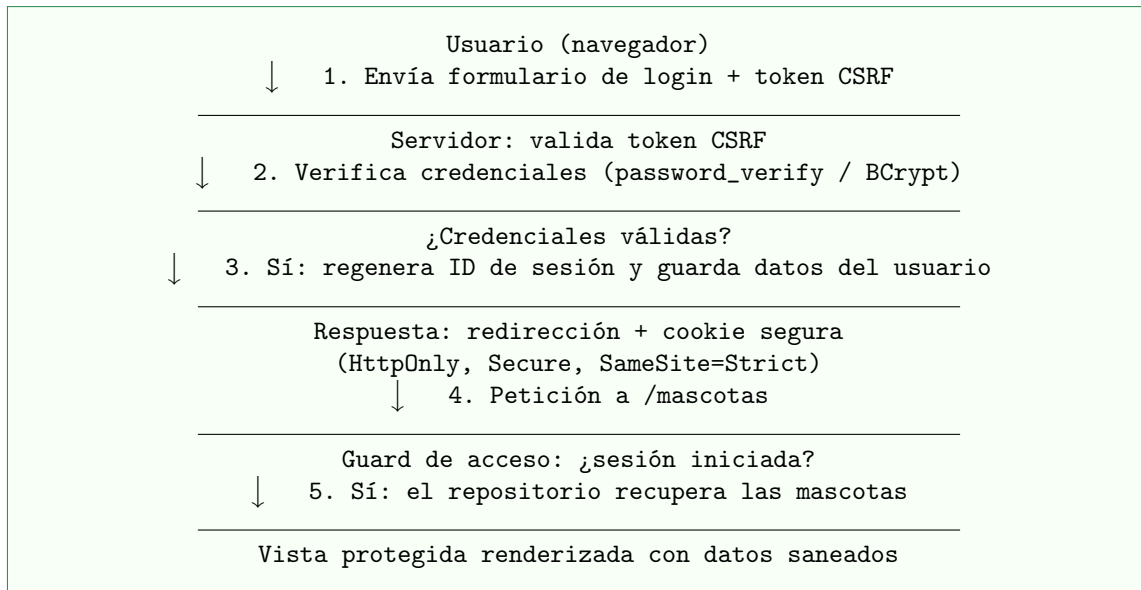
Para el contexto de la provincia de Los Ríos y de Ecuador en general, se recomienda **PHP 8.2** tanto para el PFC como para proyectos de pequeñas y medianas empresas. El factor decisivo es el costo y la disponibilidad del *hosting*: PHP funciona en cualquier plan de *hosting* compartido económico, que es lo que la mayoría de clientes locales puede costear, mientras que Spring Boot exige normalmente un VPS o un servicio en la nube, elevando el costo operativo. A esto se suma la mayor disponibilidad de desarrolladores PHP en el mercado local y su menor curva de aprendizaje, según se refleja en la tabla comparativa. Spring Boot sería la elección recomendable únicamente para proyectos empresariales de mayor escala, con presupuesto para infraestructura en la nube y requisitos de alta concurrencia sostenida.

Síntesis: resultados de PHPStan y refactorizaciones aplicadas

El análisis estático con **PHPStan en nivel 5 no reportó errores** en el código fuente del proyecto (véase la Figura 2). Durante el desarrollo, las advertencias más frecuentes que orientaron mejoras de calidad fueron los tipos de retorno no declarados y los posibles accesos a valores `null`. Las refactorizaciones aplicadas consistieron en declarar explícitamente todos los tipos de parámetros y retornos, emplear **readonly properties** para garantizar la inmutabilidad de los modelos y usar los operadores de fusión de `null` (`??`) y *nullsafe* (`?->`) de PHP 8 para el manejo seguro de valores nulos. Para detectar estos problemas antes de llegar al análisis estático, se incorporaría PHPStan a un *hook* de pre-commit de Git y a la canalización de integración continua, de modo que ningún cambio se fusione sin pasar el análisis.

Evaluación: integración con el frontend y flujo de la primera petición autenticada

El backend se integraría con el *frontend* exponiendo las vistas renderizadas en el servidor (como en esta práctica) o, en una evolución futura, una API REST que el *frontend* consumiría mediante `fetch`. El flujo de la primera petición autenticada se resume en el siguiente diagrama:



En concreto: (1) el usuario envía sus credenciales junto con el token CSRF; (2) el servidor valida el token y verifica las credenciales contra el *hash* almacenado; (3) si son válidas, regenera el identificador de sesión y almacena los datos mínimos del usuario; (4) responde con una redirección y una *cookie* de sesión endurecida; y (5) en la siguiente petición a la ruta protegida, el *guard* de acceso comprueba la sesión y, al estar autenticada, recupera las mascotas mediante el repositorio y renderiza la vista con los datos saneados.

8 Referencias

Referencias IEEE

- [1] J. Moore, M. A. Thornton, and R. W. Skeith, “Perl for Introductory Programming Courses,” Southern Methodist Univ., Dallas, TX, USA, and Univ. of Arkansas, Fayetteville, AR, USA.
- [2] F. P. E. Putra, R. W. Efendi, A. B. Tamam, and W. A. Pramadi, “Trends and Best Practices in API-Based Web Development Using Laravel and React,” *Brilliance: Research of Artificial Intelligence*, vol. 5, no. 1, pp. 223–233, May 2025, doi: 10.47709/brilliance.v5i1.5971.
- [3] V. Tiwari, M. Varshney, K. P. Singh, and S. K. Bharti, “A Comparative Study of MVC Architecture Model of Open Source Server Side Scripting Language JSP and PHP for Client-Server Architecture Based Applications Development,” *Int. J. Eng. Technol. Manage. Sci.*, vol. 8, no. 2, pp. 142–151, Mar.–Apr. 2024, doi: 10.46647/ijetms.2024.v08i02.019.
- [4] J. Turner and K. Mehta, *MySQL and JSP Web Applications: Data-Driven Programming Using Tomcat and MySQL*. Indianapolis, IN, USA: Sams Publishing.
- [5] B. Basham, K. Sierra, and B. Bates, *Head First Servlets and JSP*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media.
- [6] Tutorialspoint, “Spring Boot Tutorial.” [En línea]. Disponible: https://www.tutorialspoint.com/spring_boot/
- [7] A. Kraves and I. Ponomarev, “The Methodology for Developing Web Applications on the Platform ASP.NET Core,” *System Technologies*, vol. 1, no. 126, pp. 111–117, Mar. 2020, doi: 10.34185/1562-9945-1-126-2020-12.
- [8] *Mastering .NET Core and Entity Framework: Building Secure and Scalable Web Applications*. [En línea]. Disponible: <https://www.brightinkpublishing.com>
- [9] “The Role of the .NET Platform in Modern Web and Cloud Applications: Performance, Security, and Scalability Metrics.”
- [10] “Simplified ASP.NET Core Web API with Clean Architecture and Chain of Responsibility.”
- [11] H.-P. Halvorsen, “Web Programming — ASP.NET Core.” [En línea]. Disponible: <https://www.halvorsen.blog>
- [12] D. P. Kouru, “Design and Implementation of Secure Web Applications Using ASP.NET Core and .NET Framework,” *World Journal of Advanced Engineering Technology and Sciences*, vol. 18, no. 3, pp. 341–350, Mar. 2026, doi: 10.30574/wjaets.2026.18.3.0160.
- [13] E. B. I, “An Enhanced Mechanism for Protecting Web Applications from Cross Site Request Forgery (CSRF),” *British Journal of Computer, Networking and Information Technology*, vol. 7, no. 1, pp. 1–17, 2024, doi: 10.52589/BJCNIT_R5YYKXKA.
- [14] M. Tsaneva, “Applicability of ASP.NET Frameworks for Developing Web-Based Business Information Systems.”

Fuentes complementarias en formato APA

- 1. Moore, J., Thornton, M. A., & Skeith, R. W. *Perl for Introductory Programming Courses*. Southern Methodist University; University of Arkansas.

2. Putra, F. P. E., Efendi, R. W., Tamam, A. B., & Pramadi, W. A. (2025). Trends and best practices in API-based web development using Laravel and React. *Brilliance: Research of Artificial Intelligence*, 5(1), 223–233. <https://doi.org/10.47709/brilliance.v5i1.5971>
3. Tiwari, V., Varshney, M., Singh, K. P., & Bharti, S. K. (2024). A comparative study of MVC architecture model of open source server side scripting language JSP and PHP. *International Journal of Engineering Technology and Management Sciences*, 8(2), 142–151. <https://doi.org/10.46647/ijetms.2024.v08i02.019>
4. Basham, B., Sierra, K., & Bates, B. *Head First Servlets and JSP* (2nd ed.). O'Reilly Media.
5. Kraves, A., & Ponomarev, I. (2020). The methodology for developing web applications on the platform ASP.NET Core. *System Technologies*, 1(126), 111–117. <https://doi.org/10.34185/1562-9945-1-126-2020-12>
6. Kouru, D. P. (2026). Design and implementation of secure web applications using ASP.NET Core and .NET framework. *World Journal of Advanced Engineering Technology and Sciences*, 18(3), 341–350. <https://doi.org/10.30574/wjaets.2026.18.3.0160>